



Buffer operation support in Triton

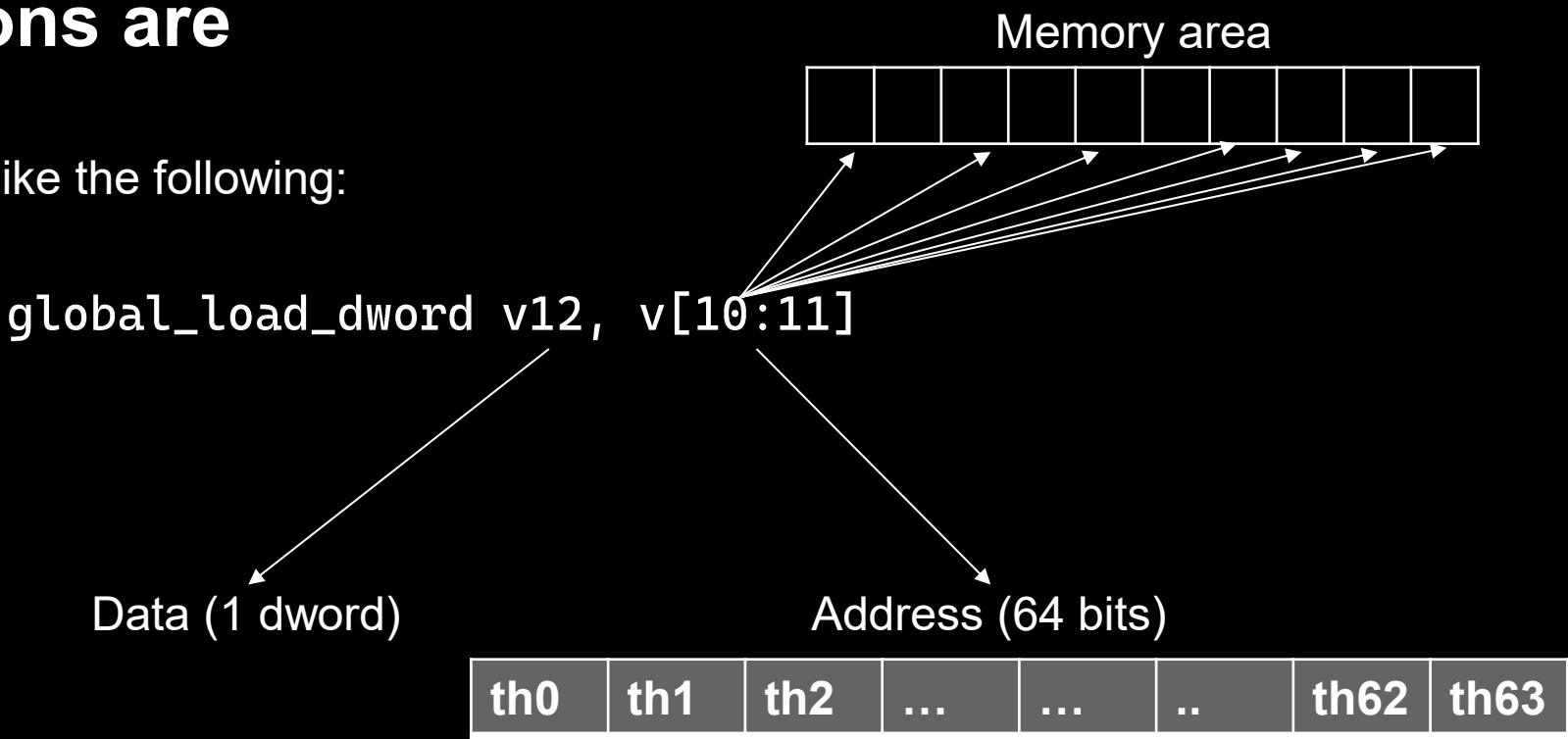
Giuseppe Rossini

Agenda

- What is a buffer operation and why bothering?
- Buffer operation support in MLIR
- Buffer operation support in Triton
 - The need of pointer canonicalization
 - Overcoming difficulties: pointer range, negativeness
 - Final outlook
- A look at performance
- Conclusion & Next steps

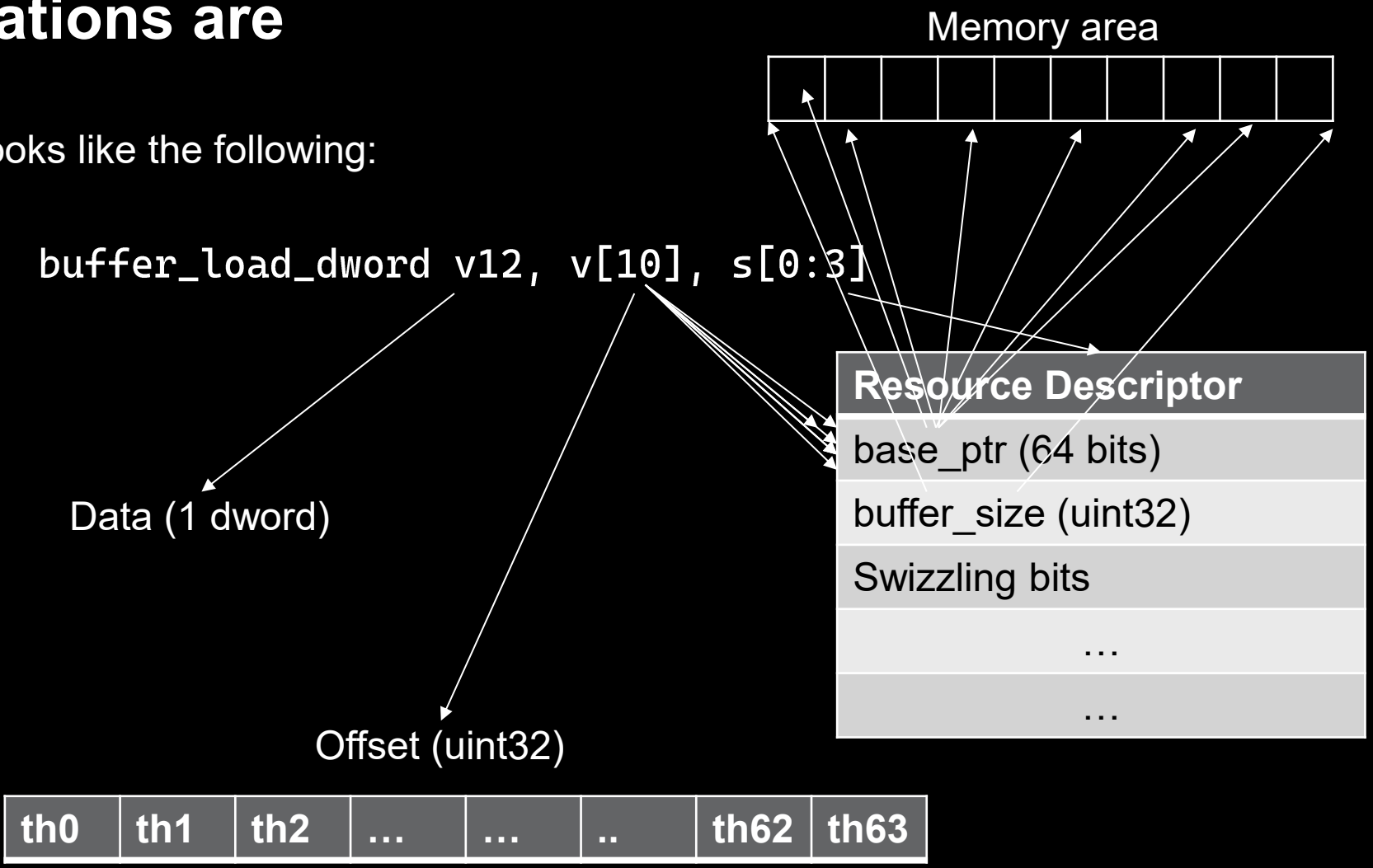
What buffer operations are

- A global_load instruction looks like the following:



What buffer operations are

- A buffer_load instruction looks like the following:



- See chapter 9 of [the MI300 ISA reference](#)

What buffer operations are

- A `buffer_load` instruction looks like the following:

```
def buffer_load_dword(offset, mem_desc)
```

Save registers
and VALU instrs

```
    vbase_address = splat(mem_desc.base_ptr)
    vaddr = vbase_address + offset
```

Can be used to
implement masking

```
    data = zeros(lanes)
    for i in lanes:
        if (offset[i]+vec_size > mem_desc.buffer_size):
            data[i] = 0
        else:
            data[i] = load vbase_address[i]
    return data
```

What buffer operations are

- How to use buffer operations to implement masking?

```
def buffer_load_dword_masked(base_ptr, offset, pred)
    mem_desc.base_ptr = base_ptr
    mem_desc.buffer_size = max_int_32
    oob_offset = max_int_32+1
    masked_offset = (pred ? offset : oob_offset)
    return buffer_load_dword(masked_offset, mem_desc)
```

→ Set the buffers as big as possible

→ Depending on the predicate use the oob offset or the real one

Buffer operation support in MLIR

- MLIR has a AMDGPU dialect that has:

```
amdgpu.raw_buffer_load %buf[%idx] : memref<?xi8> → vector<2xi8>
```

- And a straightforward conversion pass (to ROC DL):

```
matchAndRewrite() const override {
    SmallVector<Value> args;
```

```
    Value ptr = memref.alignedPtr(rewriter, loc);
    unsigned int numRecords = compute_size_bytes(memref);
```

Set the base pointer from the memref

```
    Value resource =
        rewriter.createOrFold<ROCDL::MakeBufferRsrcOp>(ptr, numRecords, ...);
    args.push_back(resource);
```

Create the res descr

```
    Value voffset = createI32Constant(rewriter, loc, 0);
    for (auto pair : llvm::enumerate(adaptor.getIndices())) {
        // Compute the offsets
    }
    args.push_back(voffset);
    rewriter.create<ROCDL::BufferOp>(loc, resultTypes, args);
```

→ Compute the offset from the memref

```
}
```

Buffer operation support in Triton

- Original estimate: 1 week.
- Simple solution: let's do what MLIR already does for AMDGPU dialect!

```
Value llLoad(Value ptr){  
  BufferEmitter bufferEmitter();
```

```
  Value offset = zero;
```

—————→ Remember? In a normal `global_load` usecase, the offset has already been added to the address!

```
  Value vecData = bufferEmitter.emitMaskedBufferLoad  
                  (elemTy, ptr, offset);  
  return vecData;  
}
```

↙
This contains address+offsets

Buffer operation support in Triton

- This works! 😊



Buffer operation support in Triton

- But performance are bad ☹️

	size	Triton	Torch
0	4096.0	4.137374	7.020711
1	8192.0	8.714893	14.124138
2	16384.0	17.066666	26.284491
3	32768.0	32.781443	50.155101
4	65536.0	60.000000	94.979713
5	131072.0	110.000000	163.490882
6	262144.0	210.000000	254.508737
7	524288.0	210.000000	344.926331
8	1048576.0	310.000000	379.918854
9	2097152.0	331.124892	390.531116
10	4194304.0	324.718216	380.493237
11	8388608.0	315.319713	391.074253
12	16777216.0	443.525879	510.253367
13	33554432.0	536.210307	616.428405
14	67108864.0	587.082978	697.967958
15	134217728.0	609.915258	755.534586



Buffer operation support in Triton

- Let me introduce you to the waterfall loop!

```

.LBB0_11:
v_readfirstlane_b32 s4, v4
v_readfirstlane_b32 s5, v5
v_readfirstlane_b32 s6, v6
v_readfirstlane_b32 s7, v7
v_cmp_eq_u64_e32 vcc_lo, s0, s1, v[4:5]
v_cmp_eq_u64_e64 s0, s1, s2, s3, s4, s5, s6, s7
s_and_b32 s0, vcc_lo, s1
s_and_saveexec_b32 s0, s1
buffer_store_b128 v[0:7], 0
s_xor_b32 exec_lo, exec_lo, s0
s_cbranch_execnz .LBB0_11

```

This is creating the resource descriptor for lane i

%cmp = %s0 < 64

Emit the buffer_store for lane i

Loop back

Buffer operation support in Triton

- We need to decouple the base address from the offset.

```
%offset0 = tt.make_range(0,256)
%ptr = tt.splat(%arg0)
%0 = tt.addptr %ptr, %offset0
%1 = tt.addptr %0, %offset0
%2 = tt.load(%1)
```



```
%offset0 = tt.make_range(0,256)
%baseOffset = triton.splat(%zero)
%0 = addi %baseOffset, %offset0
%1 = addi %0, %offset0
%2 = tt.splat(%arg0)
%3 = tt.addptr(%2, %1)
%4 = tt.load(%3)
```

Can be replaced
by a buffer load!

- I.e., we need to reconstruct the memref back!
- Other constraints stemming from this decoupling
 - The offsets need to be representable as int32 (i.e., the size of the memory tensor cannot be bigger than 2GB)
 - The offsets cannot be negative

Pointer canonicalization

- Internally, we have an internal struct that basically is a simplified version of memref:

```
struct FatPtr{  
    Value basePtr; // scalar base pointer  
    Value offset; // tensor offset (each different for each thread)  
}
```

- Each global pointer (i.e., pointers passed as argument to the Triton kernel) will have `FatPtr` associated with it.
- The association is stored into a table
$$\text{Pointer} = \text{map}\langle \text{Value}, \text{FatPtr} \rangle$$
- We need to walk through all the uses of a given global pointer
- PR: <https://github.com/triton-lang/triton/pull/4241>

Pointer canonicalization

```
%newPtr = tt.addptr %ptr, %offset
```



```
%newOffset = addi Pointer[%ptr].offset, %offset  
Pointer[%newPtr] = {Pointer[%ptr].basePtr, %newOffset}  
for (Use value : %newPtr.getUses()){  
    // keep going  
}
```

Pointer canonicalization

```
%newPtr = tt.addptr %scalarPtr, %scalarOffset
```



```
Pointer[%newPtr].basePtr = %newPtr  
Pointer[%newPtr].offset = Pointer[%scalarPtr].offset  
for (Use value : %newPtr.getUses()){  
    // keep going  
}
```

Pointer canonicalization

```
%data = tt.load %ptr
```



Pointer materialization

```
%splatPtr = tt.splat(%Pointer[%ptr].basePtr)  
%newPtr = addi %splatPtr, %Pointer[%ptr].offset  
data = tt.load(%newPtr)
```


32-bit pointer range

- The offset field in a buffer operation needs to be uint32.
- However, to apply the “mask trick” we need the offset to be int32 (because we use the other “half” of the 32-bit range to say that the offset is out-of-range)
- Solution 1) Range analysis
 - Pros: super generic
 - Cons: does not work in too many cases. In many cases each thread of a workgroup is multiplied by a 32-bit stride. 32-bit * 32-bit is 64 bit. No good.
- Solution 2) Range analysis at LLVM level
 - Pros: still super generic
 - Pros: at LLVM we can “break” the workgroup in waves, whose threads are usually coalesced. There is no multiplication by any stride at wave level
 - Cons: harder because in LLVM
- ✓ Solution 3) Automatically infer the size of the input tensors from what the user passes at runtime:
 - Pros: is quite generic
 - Cons: it might increase recompilation

32-bit pointer range

- What happens when the user calls a kernel?

```
add_kernel[grid](x, y, output, n_elements, BLOCK_SIZE=1024)
```



```
backend.get_attrs_descriptor(self.params, values)
```



```
# tt.pointer_range: does the pointer space fit in 2GB
self.arg_properties["tt.pointer_range"] =
    [ param.num for param, arg in zip(params, values)    if is_within2gb(arg)]
```



```
tt.func public @add_kernel(%arg0: !tt.ptr<f32> {tt.divisibility = 16 : i32, tt.pointer_range = 32 : i32}, ...)
```



Cached



- PR: <https://github.com/triton-lang/triton/pull/4910>

Negativeness

- Can we use the same trick?
- Yes, but now if there are K integers passed to a kernel, we might have 2^K combinations!
- In practice, this never happens (think about GEMM when any of the strides is negative?)
- However, even if bit the bullet and added negativeness as an “inferred” property, in many cases we could still not guarantee that a scalar is negative

```
pid = tl.program_id(axis=0)
num_pid_m = tl.cdiv(M, BLOCK_SIZE_M)
num_pid_n = tl.cdiv(N, BLOCK_SIZE_N)
num_pid_in_group = GROUP_SIZE_M * num_pid_n
group_id = pid // num_pid_in_group
first_pid_m = group_id * GROUP_SIZE_M
group_size_m = min(num_pid_m - first_pid_m, GROUP_SIZE_M)
pid_n = (pid % num_pid_in_group) // group_size_m
```

Very hard to determine if this is non-negative

Negativeness

- Solution proposed: we need to stick to assumptions
- The user will have to tell us that the scalars in the kernel are non-negative.
- We might improve this, but it's not a simple problem (of course)

```
tl.assume(stridea_m > 0)
tl.assume(strideb_k > 0)
tl.assume(stridec_m > 0)
tl.assume(stridec_n > 0)
pid = tl.program_id(axis=0)
num_pid_m = tl.cdiv(M, BLOCK_SIZE_M)
num_pid_n = tl.cdiv(N, BLOCK_SIZE_N)
num_pid_in_group = GROUP_SIZE_M * num_pid_n
group_id = pid // num_pid_in_group
first_pid_m = group_id * GROUP_SIZE_M
group_size_m = min(num_pid_m - first_pid_m, GROUP_SIZE_M)
pid_n = (pid % num_pid_in_group) // group_size_m
tl.assume(pid_n > 0)
```

Final outlook

- We introduced two new operations in the AMD flavour of the ttgir dialect:

```
amdgpu.buffer_load %basePtr[%offset], %mask  
amdgpu.buffer_store %value, %basePtr[%offset], %mask
```

- Compilation flow:

```
amd.passes.ttgpuir.add_canonicalize_pointers
```



```
amd.passes.ttgpuir.add_convert_to_buffer_ops
```



```
amd.passes.ttgpuir.add_to_llvmir
```

- PR: <https://github.com/triton-lang/triton/pull/4903>

Final outlook

Base function

```
tt.func @test(%arg0: !tt.ptr<f32>{tt.pointer_range = 32 : i32}){
  %c1024_i32 = arith.constant 1024 : i32
  %0 = tt.get_program_id x : i32
  %1 = arith.muli %0, %c1024_i32 : i32
  %2 = tt.make_range {end = 1024 : i32, start = 0 : i32} : tensor<1024xi32, #blocked>
  %3 = tt.splat %1 : i32 -> tensor<1024xi32, #blocked>
  %4 = arith.addi %3, %2 : tensor<1024xi32, #blocked>
  %5 = tt.splat %arg0 : !tt.ptr<f32> -> tensor<1024x!tt.ptr<f32>, #blocked>
  %6 = tt.addptr %5, %4 : tensor<1024x!tt.ptr<f32>, #blocked>, tensor<1024xi32, #blocked>
  %7 = tt.addptr %6, %4 : tensor<1024x!tt.ptr<f32>, #blocked>, tensor<1024xi32, #blocked>
  %8 = tt.load %7 : tensor<1024x!tt.ptr<f32>, #blocked>
}
```

↓
We need to canonicalize this before emitting the buffer load

Final outlook

After `-tritonamdgpu-canonicalize-pointers`

```
tt.func @test(%arg0: !tt.ptr<f32> {tt.pointer_range = 32 : i32}) {
  %c1024_i32 = arith.constant 1024 : i32
  %0 = tt.get_program_id x : i32
  %1 = arith.muli %0, %c1024_i32 : i32
  %2 = tt.make_range {end = 1024 : i32, start = 0 : i32} : tensor<1024xi32, #blocked>
  %3 = tt.addptr %arg0, %1 : !tt.ptr<f32>, i32
  %4 = tt.addptr %3, %1 : !tt.ptr<f32>, i32
  %5 = arith.addi %2, %2 : tensor<1024xi32, #blocked>
  %6 = tt.splat %4 : !tt.ptr<f32> -> tensor<1024x!tt.ptr<f32>, #blocked>
  %7 = tt.addptr %6, %5 : tensor<1024x!tt.ptr<f32>, #blocked>, tensor<1024xi32, #blocked>
  %8 = tt.load %7 : tensor<1024x!tt.ptr<f32>, #blocked>
}
```

Scalar updates

Non-scalar updates

Ready for buffer load!

Final outlook

After `-tritonamdgpu-convert-buffer-ops`

```
tt.func @test(%arg0: !tt.ptr<f32> {tt.pointer_range = 32 : i32}) {  
    %c1024_i32 = arith.constant 1024 : i32  
    %0 = tt.get_program_id x : i32  
    %1 = arith.muli %0, %c1024_i32 : i32  
    %2 = tt.make_range {end = 1024 : i32, start = 0 : i32} : tensor<1024xi32, #blocked>  
    %3 = tt.addptr %arg0, %1 : !tt.ptr<f32>, i32  
    %4 = tt.addptr %3, %1 : !tt.ptr<f32>, i32  
    %5 = arith.addi %2, %2 : tensor<1024xi32, #blocked>  
    %6 = amdgpu.buffer_load %4[%5] : tensor<1024xf32, #blocked>  
}
```



Final outlook

After -convert-triton-amdgpu-to-llvm

```

llvm.func @test(...) {
    // ...
    %7 = rocdl.make.buffer.rsrc %arg0, %4, %6, %5 : <1> to <8>
    %8 = llvm.mlir.constant(true) : i1
    %9 = llvm.mlir.constant(dense<0.000000e+00> : vector<4xf32>) : vector<4xf32>
    %10 = llvm.mlir.constant(-2147483648 : i32) : i32
    %11 = llvm.mlir.constant(4 : i32) : i32
    %12 = llvm.mul %11, %0 : i32
    %13 = llvm.select %8, %12, %10 : i1, i32
    %14 = llvm.mlir.constant(0 : i32) : i32
    %15 = llvm.mlir.constant(0 : i32) : i32
    %16 = rocdl.raw.ptr.buffer.load %7, %13, %14, %15 : vector<4xf32>
    // ...
}

```

Mask logic

A look at performance

- This is only a glimpse of what performance might look like
 - Gfx11 performance
 - Extracted from tutorial
- On non-power-of-two the reduction of the number of branches brings up to 40% improvement
- On power-of-two the decrease of VALU instructions and register pressure gives up to 17% improvement
- More detailed analysis needs to be fulfilled

M/N/K	new	baseline	% improvements
259	2.341507	2.024821	15.64019733
387	6.300065	5.43695	15.87498506
515	12.10912	10.178158	18.97162532
643	18.98912	15.976185	18.85892032
771	23.84506	21.496905	10.92323755
899	27.77366	26.363067	5.350625555
1027	30.12213	25.427388	18.46331208
1155	30.48642	25.921701	17.60964298
1283	37.03812	27.391761	35.21628274
1411	42.44707	27.278716	55.6050952
1539	39.12736	32.280566	21.21026007
1667	39.65373	30.041262	31.99754724
1795	43.76443	31.144254	40.52168018
1923	44.64497	34.856367	28.08267138
2051	42.71962	33.086576	29.11464456
2179	46.33169	33.935382	36.52914236
2307	45.8111	36.847454	24.32635916
2435	45.74041	34.339086	33.20217958
2563	45.09224	36.838872	22.40395146
2691	45.6939	36.07569	26.66118929
2819	45.29983	37.180037	21.83911221
2947	48.94033	39.437701	24.0953016
3075	46.84344	38.00259	23.26379597
3203	46.9776	37.678217	24.68106705
3331	47.71292	38.923431	22.58149596
3459	47.36687	37.64537	25.82388751
3587	48.73585	39.76011	22.57472628
3715	47.38801	38.634482	22.65729355
3843	48.35917	39.390743	22.7678569
3971	46.29512	39.284955	17.84439361
4099	50.3508	40.0679	25.66368839

Non-power-of-2 dims

M/N/K	new	Baseline	% improvement
256	2.964173	2.573193	15.19435192
384	8.302508	7.060238	17.5952992
512	15.148727	13.61235	11.28667314
640	25.750884	22.75556	13.16306722
768	33.804839	31.13244	8.583958541
896	44.076174	43.38368	1.596201503
1024	43.52419	42.37256	2.717867412
1152	61.546854	56.93891	8.092795251
1280	58.44824	57.08012	2.396850462
1408	65.004116	64.19681	1.257553199
1536	68.986509	70.28469	-1.847033801
1664	73.087028	74.52937	-1.935267909
1792	72.094076	71.46733	0.876974185
1920	81.261165	74.71585	8.760283712
2048	81.266735	74.65548	8.855680314
2176	81.875599	75.16767	8.923953491
2304	84.122206	81.63469	3.047133591
2432	82.668296	79.33072	4.207164533
2560	83.743297	77.44927	8.126640517
2688	83.200958	78.3063	6.250652645
2816	83.396726	77.26741	7.932606047
2944	82.29072	77.3114	6.440603927
3072	82.774282	80.20459	3.203926519
3200	83.404072	78.29425	6.526437377
3328	84.433239	78.60112	7.419892999
3456	84.845094	79.19167	7.138913817
3584	85.67139	80.01579	7.068110285
3712	84.677829	81.52474	3.867645529
3840	85.824535	82.70058	3.777424399
3968	84.344793	80.52859	4.738939183
4096	86.72486	81.74297	6.094577844

Power-of-2 dims

Conclusion and future work

- This work introduces buffer operations in Triton
- I introduced the basic pieces:
 - Pointer canonicalization
 - Pointer range inference
 - Basic conversions (tt.load -> amdgpu.buffer_load -> rocdl.buffer_load)
- More work needs to be carried on
 - Full performance assessment <https://github.com/ROCm/triton-internal/issues/379> and <https://github.com/ROCm/triton-internal/issues/178>
 - Canonicalization correctness <https://github.com/ROCm/triton-internal/issues/352>
 - Canonicalization design improvements: <https://github.com/ROCm/triton-internal/issues/380>
 - Cache modifiers : <https://github.com/ROCm/triton-internal/issues/368>
 - Add more buffer operations (atomic add, atomic cmp&swap, etc...): <https://github.com/ROCm/triton-internal/issues/381>
 - Add more analysis in LLVM: <https://github.com/ROCm/triton-internal/issues/187>

AMD

